

DeepSeek V4 推理性能分析：Ascend 910C vs NVIDIA H20

1. 摘要

本报告基于 Roofline 模型，对 DeepSeek V4 在 Ascend 910C 和 NVIDIA H20 硬件平台上的推理性能进行了全面分析，涵盖瓶颈分析、参数优化、P/D 分离部署和跨平台对比，覆盖四种上下文长度场景（8K、32K、128K、256K 输入 + 4K 输出）。

核心发现：

1. KV 压缩是 V4 的决定性优势。异构压缩调度（2 个全注意力层 + 21 个 C4A 层 + 20 个 C128A 层）在 8K 下将 KV 缓存压缩至每 batch 元素仅 0.062 GB——比 V3 的 MLA 小 9.3 倍，在 128K 下比未压缩小 12.7 倍。这使得在相同 HBM 预算下可支持 10 倍更大的 batch 规模。
 2. mHC 内核融合是实际部署的必要条件。未融合时 mHC 消耗 84.6% 的 Prefill 时间。融合 BF16 内核配合序列并行将其降至 3.5%，带来 6.25 倍的端到端 Prefill 加速。
 3. 910C 与 H20 具有互补优势。910C 实现 1.68--1.78 倍更快的 Prefill 延迟（计算优势），而 H20 实现 2.13--2.14 倍更快的 Decode（内存带宽优势）。在 Decode 吞吐方面，910C 在 8K 下领先（1.14 倍），得益于其 7.84 倍的 EP 带宽优势，但 H20 在 32K+ 下占据主导（256K 下高达 2.77 倍）。
 4. P/D 分离在长上下文下变得不可或缺。P:D GPU 比例从 8K 的 1:1 增长到 256K 时 910C 的 3:1 和 H20 的 14:1。H20 在 256K 时极端的 14:1 比例需要 144 个 GPU，而 910C 仅需 80 个，使 910C 在超长上下文下更具成本效益。
 5. Lightning Index 在长上下文下成为主要开销。在 910C 上从 8K 时 Prefill 时间的 4.4% 上升至 256K 时的 44.4%，Lightning Index 超越 mHC 成为首要瓶颈——这是下一步优化的前沿方向。
 6. EP 网络带宽决定 MoE 服务经济性。910C 的 392 GB/s EP 带宽（H20 的 50 GB/s 的 7.84 倍）彻底改变了 MoE 通信格局：通信占 H20 Decode 时间的 57.3%，而在 910C 上仅占 21.6%，使数据中心网络设计成为 MoE 部署的首要考量。
-

2. 模型架构

2.1 DeepSeek V4 架构

DeepSeek V4 代表了 DeepSeek 模型家族的根本性重新设计，以原始参数规模换取推理效率。V4 总参数量约 285B——不到 V3 的 704B 的一半——通过三大关键架构创新实现了有竞争力的模型质量：

MQA 配合 KV 压缩。 V4 放弃了 V3 的多头潜在注意力（MLA），转而采用多查询注意力（MQA，64 个 Q 头共享 1 个 KV 头）并结合激进的 KV 压缩。43 层使用异构压缩调度：2 个全注意力层（ratio=1）、21 个 C4A 层（4 倍压缩）和 20 个 C128A 层（128 倍压缩）。这使得每 token 仅需 15,168 字节的 KV 缓存——相比 V3 每 token 70,272 字节减少了 4.6 倍。

Lightning Index。 为补偿激进 KV 压缩带来的信息损失，V4 引入了学习型稀疏检索机制，使用 64 个索引头（dim=128），在每个压缩注意力层中为每个查询选择 top-512 个最相关的 KV 条目。这使模型在保持小缓存占用的同时，维持了长上下文任务的质量。

manifold Hyper Connection (mHC)。 V4 用超级连接机制（hc_mult=4）替代标准残差连接，在层边界处将隐藏状态扩展 4 倍，以改善梯度流动和跨层信息共享。虽然功能强大，但这导致中间激活内存 4 倍放大，使 mHC 成为一个显著瓶颈，详见第 5.1 节讨论。

参数	值
Hidden size	4,096
Layers	43 (2 full + 21 C4A + 20 C128A)
Q heads	64 (MQA: 1 KV head)
Head dim	512
Q LoRA rank	1,024
O groups	8, O LoRA rank = 1,024
Index heads	64, dim = 128, topK = 512
Window size	128 (SWA)
Routed experts	256, top-6
Shared experts	1
MoE inter dim	2,048
HC mult	4
Vocab size	129,280

2.2 V4 与 V3 对比

V4 的设计理念是"更小的模型，更智能的推理"。以下对比展示了其中的权衡：

维度	DeepSeek V4	DeepSeek V3
Hidden size	4,096	7,168
Layers	43	61
Q heads	64	128
KV 方案	MQA + KV compression	MLA (kv_lora_rank=512)
Q LoRA rank	1,024	1,536
Routed experts	256, top-6	256, top-8
MoE inter dim	2,048	2,048
KV compression	C4A/C128A (2--128x)	无
mHC (manifold Hyper Connection)	有 (hc_mult=4)	无
Lightning Index	有 (64 heads, dim=128)	无
总参数量 (约)	~285B	~704B
每 token KV 缓存	15,168 bytes	70,272 bytes
KV 压缩比	比 V3 少 4.6 倍	基线
Attn FLOPs per token	226,492,416	374,210,560
MoE FLOPs per token	352,321,536	792,723,456
Attn params per layer	113,246,208	187,105,280
MoE params per layer	6,468,665,344	11,320,164,352

最显著的差异是总参数量 2.5 倍的减少（285B vs 704B），这直接转化为更低的权重内存需求——在 TP=8、EP=16 配置下，V4 每 rank 约需 40 GB（BF16）。V4 每 token 的注意力 FLOPs 降低 1.65 倍（226M vs 374M），MoE FLOPs 降低 2.25 倍（352M vs 793M），这得益于更少的激活专家数（top-6 vs top-8）和更小的隐藏维度。KV 缓存的缩减更为显著：V4 每 token 仅存储 15,168 字节，而 V3 为 70,272 字节——改善了 4.6 倍——使 V4 在相同 HBM 预算下能服务远更大的 batch 规模。这是 V4 在内存受限部署中实现更优单 GPU 吞吐的关键推动因素。

2.3 硬件平台

两个目标平台具有截然不同的硬件特性，导致不同的推理特征：

指标	Ascend 910C	NVIDIA H20	比率
Cube TFLOPS (BF16)	376	148	910C 2.54x
Vec TFLOPS (FP32)	24	44	H20 1.83x
Cube:Vec 比率	15.7:1	3.4:1	H20 更均衡
HBM 带宽 (GB/s)	1,800	4,000	H20 2.22x
HBM 容量 (GB)	64	96	H20 1.5x
TP 带宽 (GB/s)	392	450	接近
EP 带宽 (GB/s)	392	50	910C 7.84x
Flops 利用率	50%	50%	相同
HBM BW 利用率	80%	80%	相同

910C 优势：计算与 EP 互联。910C 的 BF16 矩阵乘法吞吐量是 H20 的 2.54 倍（376 vs 148 TFLOPS），使其在计算密集型 Prefill 工作负载上显著更快。关键的是，910C 提供 392 GB/s 的 EP 带宽——H20 的 50 GB/s 的 7.84 倍——因为其节点间网络使用与节点内 TP 相同的高带宽互联。这使 MoE 专家并行通信在 910C 上的成本大幅降低，直接有利于 256 专家 MoE 所需的 all-to-all 调度。

H20 优势：内存带宽与容量。H20 的 4,000 GB/s HBM 带宽（910C 1,800 GB/s 的 2.22 倍）在 Decode 中占主导地位的内存密集型操作上具有决定性优势：加载 MoE 专家权重、读取 KV 缓存和执行 mHC 激活操作。H20 还拥有 50% 更大的 HBM 容量（96 GB vs 64 GB），支持更大的 batch 规模或更长的序列。其 Cube:Vec 比率为 3.4:1（910C 为 15.7:1），表明架构更为均衡，向量化操作受到的相对惩罚更小。

综合效果。910C 凭借其在矩阵乘法密集型 Prefill 上的计算优势，赢得 Prefill 延迟（1.68--1.78 倍更快，见表 4.4）。H20 赢得 Decode 延迟（2.13--2.14 倍更快），因为 Decode 由 MEM 受限的权重和 KV 缓存读取主导。对于吞吐工作负载，情况更为复杂，高度依赖上下文长度，详见第 4 节和第 6 节。

3. 瓶颈分析

3.1 各类别瓶颈总结

Prefill 和 Decode 的瓶颈分布截然不同。Prefill 由 CUBE（矩阵乘法）操作主导，覆盖注意力投影、KV 压缩和 MoE，因为较大的序列维度产生较高的算术强度。Decode 每次处理单个 token，几乎所有类别都转为 MEM 受限（内存带宽）——通信除外，它在两个阶段都保持 COMM 受限。唯一在两个阶段都是 MEM 受限的类别是 mHC，因为融合内核的主要开销是读写 4 倍扩展的激活，而非执行矩阵乘法。

类别	Prefill 瓶颈	Decode 瓶颈
注意力投影	CUBE	MEM
注意力计算	CUBE	MEM
KV 压缩	CUBE	MEM
Lightning Index	CUBE/COMM	COMM/MEM
mHC（融合）	MEM	MEM
MoE Gate	CUBE	MEM
MoE Routed	CUBE	MEM
通信	COMM	COMM

3.2 Prefill 算子分布（910C，最优吞吐量配置，融合）

在 910C 上使用最优吞吐量配置（TP=8，EP=16，DP=2）和融合 mHC 内核时，Prefill 瓶颈格局随上下文长度显著演变：

类别	8K/4K	32K/4K	128K/4K	256K/4K
mHC	36.8%	34.0%	25.8%	19.6%
注意力投影	23.2%	21.4%	16.3%	12.3%
Lightning Index	4.4%	11.2%	30.0%	44.4%
通信	18.3%	16.9%	12.9%	9.7%
KV 压缩	7.4%	6.9%	5.2%	4.0%
注意力计算	4.5%	4.7%	6.1%	7.2%

类别	8K/4K	32K/4K	128K/4K	256K/4K
MoE (Gate+Routed)	3.1%	2.8%	2.2%	1.6%
Norm	0.3%	0.3%	0.2%	0.2%
Embedding/LMHead	1.9%	1.8%	1.3%	1.0%

在 8K 上下文下，mHC 以 36.8% 占据主导，其次是注意力投影（23.2%）和通信（18.3%）。即使经过内核融合，mHC 的 4 倍激活扩展仍产生大量 HBM 流量，910C 的 1,800 GB/s 带宽无法完全隐藏。随着上下文长度增长，出现关键转变：Lightning Index 从 8K 的 4.4% 上升至 256K 的 44.4%，超越 mHC 成为主要开销。这是因为 Lightning Index 涉及的矩阵乘法大小随序列长度增长（在完整 KV 缓存上计算索引分数），而 mHC 开销与序列长度无关。相应地，mHC 的占比从 36.8% 降至 19.6%，因为其绝对值保持不变。MoE 始终占较小比例（1.6--3.1%），因为共享专家与调度通信重叠，且 EP=16 将 256 个路由专家分布在足够多的 rank 上，使每 rank 专家权重加载可控。

3.3 Decode 算子分布（910C，最优吞吐量配置）

Decode 呈现与 Prefill 根本不同的分布，这源于单 token 处理使算术强度骤降：

类别	8K/4K	32K/4K	128K/4K	256K/4K
MoE Routed	46.6%	34.7%	44.7%	46.4%
通信	21.6%	26.2%	14.8%	14.3%
Lightning Index	10.1%	20.9%	26.3%	27.2%
mHC	6.8%	5.0%	1.6%	0.8%
注意力计算	6.6%	5.9%	2.5%	1.3%
注意力投影	4.3%	3.8%	7.2%	7.4%
KV 压缩	3.0%	2.6%	1.8%	1.7%
MoE Gate	0.3%	0.3%	0.3%	0.3%
Norm	0.5%	0.4%	0.1%	0.1%
Embedding/LMHead	0.4%	0.3%	0.7%	0.7%

MoE 路由专家在所有上下文长度下以 34.7--46.6% 的占比主导 Decode。在 batch=512、DP=2 配置下，每个 rank 处理 256 个请求，但每步每层仅从 16 个本地专家（EP=16）中加载 6 个专家，产生每步每层 24 MB 的权重读取——纯内存带宽负载。通信是第二大组成部分（14.3--26.2%），包括 MoE all-to-all 调度/合并以及 SP all-gather 操作。Lightning Index 从 8K 的 10.1% 增长到 256K 的

27.2%，因为索引键读取随总序列长度增长。值得注意的是，mHC 在 256K 的 Decode 中降至不到 1%——每 token 的 mHC 开销很小，因为它仅对单个 token 进行 4 倍扩展处理，不像 Prefill 需要处理整个 batch 乘以序列长度。

3.4 910C vs H20 瓶颈对比

910C 和 H20 在每个类别上呈现相同的瓶颈类型（两者 Prefill 矩阵乘法均为 CUBE 受限，Decode 权重加载均为 MEM 受限），但由于计算带宽比不同，相对比例差异显著：

Prefill 瓶颈对比（8K/4K）：

类别	910C	H20	910C 瓶颈	H20 瓶颈
mHC	36.8%	9.2%	MEM	MEM
注意力投影	23.2%	32.9%	CUBE	CUBE
通信	18.3%	27.0%	COMM	COMM
KV 压缩	7.4%	10.5%	CUBE	CUBE
Lightning Index	4.4%	5.1%	CUBE	CUBE
注意力计算	4.5%	5.8%	CUBE	CUBE
MoE Routed	1.8%	4.9%	CUBE	CUBE
MoE Gate	1.3%	1.8%	CUBE	CUBE
Embedding/LMHead	1.9%	2.7%	CUBE	CUBE

Decode 瓶颈对比（8K/4K）：

类别	910C	H20	910C 瓶颈	H20 瓶颈
MoE Routed	46.6%	18.4%	MEM	MEM
通信	21.6%	57.3%	COMM	COMM
Lightning Index	10.1%	5.0%	COMM	COMM
mHC	6.8%	2.7%	MEM	MEM
注意力计算	6.6%	2.6%	MEM	MEM
注意力投影	4.3%	9.5%	CUBE	CUBE
KV 压缩	3.0%	3.1%	MEM	CUBE

Prefill：在 910C 上 mHC 是最大开销，占 36.8%，而 H20 仅 9.2%。这 4 倍的差异源于 mHC 是 MEM 受限的，H20 的 2.22 倍更高的 HBM 带宽能更快地完成这些读取——虽然 910C 的矩阵乘法密集型操作（910C 上注意力投影占 23.2%）因 2.54 倍的计算优势而运行更快，但 MEM 受限的 mHC 无法被隐藏。在 H20 上，注意力投影（32.9%）和通信（27.0%）是主要开销。通信在 H20 上比例更大，因为其较慢的 EP 带宽（50 GB/s vs 392 GB/s）增大了 all-to-all 时间。

Decode：最显著的对比是通信：H20 Decode 时间的 57.3% 对比 910C 的 21.6%。H20 的 50 GB/s EP 带宽在 MoE 调度/合并每步需要在 16 个 rank 间传输专家激活时成为关键瓶颈。910C 的 392 GB/s EP 带宽（快 7.84 倍）使 MoE 通信相对廉价，因此 MoE 权重读取反而占据主导（46.6% vs 18.4%）。EP 带宽差异是解释 910C 在短上下文下尽管 HBM 带宽低 2.22 倍仍能实现有竞争力的 Decode 吞吐的最重要因素。

4. 参数与场景优化

4.1 最优配置——Ascend 910C

在 910C 上进行 TP 属于 {1..64}、EP 属于 {1..256}、DP 属于 {1..8}、BS 属于 {1..512} 的网格搜索，揭示了一致的模式：延迟最优配置最大化并行度（TP=8，EP=64，DP=8）以最小化每 rank 工作量，而吞吐最优配置最小化 GPU 数量（TP=8，EP=16，DP=2）并最大化 batch 规模以分摊固定开销。值得注意的是，Prefill 吞吐配置在所有上下文长度下稳定保持在 TP=8、EP=16、DP=2，因为 910C 的计算优势意味着 Prefill 随 batch 规模线性扩展，直到 HBM 容量限制最大 batch。Decode 延迟配置一致使用 TP=4、EP=32、DP=8——比 Prefill 更低的 TP，因为 Decode 是内存受限而非计算受限的，TP=4 提供足够的并行度而无需 TP=8 的通信开销。

8K 输入 / 4K 输出

场景	TP	EP	DP	BS	GPU 数	指标
Prefill 延迟	8	64	8	8	64	325 ms
Prefill 吞吐量	8	16	2	512	16	1,679 tps/gpu
Decode 延迟	4	32	8	8	32	19.4 ms/step
Decode 吞吐量	8	16	2	512	16	307 tps/gpu

32K 输入 / 4K 输出

场景	TP	EP	DP	BS	GPU 数	指标
Prefill 延迟	8	64	8	8	64	1,327 ms
Prefill 吞吐量	8	16	2	128	16	1,551 tps/gpu
Decode 延迟	4	32	8	8	32	19.4 ms/step
Decode 吞吐量	8	32	4	512	32	115 tps/gpu

128K 输入 / 4K 输出

场景	TP	EP	DP	BS	GPU 数	指标
Prefill 延迟	8	64	8	8	64	6,914 ms
Prefill 吞吐量	8	16	2	32	16	1,179 tps/gpu
Decode 延迟	4	32	8	8	32	20.8 ms/step
Decode 吞吐量	4	32	8	256	32	37.2 tps/gpu

256K 输入 / 4K 输出

场景	TP	EP	DP	BS	GPU 数	指标
Prefill 延迟	8	64	8	8	64	18,255 ms
Prefill 吞吐量	8	16	2	16	16	893 tps/gpu
Decode 延迟	4	32	8	8	32	21.0 ms/step
Decode 吞吐量	4	32	8	128	32	19.3 tps/gpu

4.2 最优配置——NVIDIA H20

H20 的最优配置揭示了一种明显不同的策略，这是由其带宽充裕、计算有限的硬件特性和窄 EP 互联所驱动的。Prefill 吞吐配置一致稳定在 TP=8、EP=8、DP=1（仅 8 个 GPU）——这是能将模型权重放入 H20 的 96 GB HBM（EP=8 时权重 74.5 GB）的最小 EP 配置。之所以最优，是因为 H20 的 50 GB/s EP 带宽使更宽的 EP 拆分在通信开销上代价过高；保持 EP=8 最小化了 all-to-all 传输。对于 Decode 吞吐，H20 倾向于 TP=4 配合适度的 EP（16--32），因为随上下文长度增长，4,000 GB/s HBM 带宽即使在较少 EP rank 下也能高效服务 MoE 权重读取。Decode 延迟在 8K--256K 范围内显著稳定，保持在 9.1--9.9 ms，反映了 H20 的内存带宽能高效处理 KV 缓存增长。

8K 输入 / 4K 输出

场景	TP	EP	DP	BS	GPU 数	指标
Prefill 延迟	8	64	8	8	64	547 ms
Prefill 吞吐量	8	8	1	256	8	1,872 tps/gpu
Decode 延迟	4	32	8	8	32	9.1 ms/step
Decode 吞吐量	8	16	2	512	16	269 tps/gpu

32K 输入 / 4K 输出

场景	TP	EP	DP	BS	GPU 数	指标
Prefill 延迟	8	64	8	8	64	2,345 ms
Prefill 吞吐量	8	8	1	64	8	1,726 tps/gpu
Decode 延迟	4	32	8	8	32	9.1 ms/step
Decode 吞吐量	4	16	4	512	16	205 tps/gpu

128K 输入 / 4K 输出

场景	TP	EP	DP	BS	GPU 数	指标
Prefill 延迟	8	64	8	8	64	12,328 ms
Prefill 吞吐量	8	8	1	16	8	1,314 tps/gpu
Decode 延迟	4	32	8	8	32	9.8 ms/step
Decode 吞吐量	4	16	4	128	16	91.1 tps/gpu

256K 输入 / 4K 输出

场景	TP	EP	DP	BS	GPU 数	指标
Prefill 延迟	8	64	8	8	64	32,581 ms
Prefill 吞吐量	8	8	1	8	8	997 tps/gpu
Decode 延迟	4	32	8	8	32	9.9 ms/step
Decode 吞吐量	4	32	8	256	32	53.5 tps/gpu

4.3 P/D 分离式服务

P/D 分离将 Prefill 和 Decode 拆分为独立扩展的集群，每个集群使用各自的最优配置。P:D 比例（每个 Decode 实例对应的 Prefill 实例数）由匹配两者吞吐量来确定： $P:D =$

$\text{ceil}(\text{prefill_tps_instance} / \text{decode_tps_instance} * \text{output_len} / \text{input_len})$ 。

在 910C 上，P:D 比例从 8K 的 1:1 适度增长到 256K 的 3:1。这反映了 910C 均衡的 EP 带宽：Prefill 吞吐从 26,861 降至 14,291 tps/实例（随上下文增长），而 Decode 吞吐从 4,913 降至 618 tps/实例。比例保持可控，因为 910C 的快速 EP 互联使 Prefill 和 Decode 的通信开销保持成比例。

在 H20 上，P:D 比例增长更为激进：从 8K 的 1:1 到 256K 的 14:1（共 144 个 GPU）。这种极端比例的出现是因为 H20 在 256K 下的 Decode 吞吐（32 GPU 上 1,711 tps/实例）相对于 Prefill（8 GPU 上 7,974 tps/实例）出人意料地高，但 64:1 的输入输出比意味着每个解码 token 的"代价"是 64 个预填充 token。H20 需要 14 个 Prefill 实例才能保持一个 Decode 实例饱和。这对集群规模有重要影响：超长上下文在 H20 上需要严重偏向 Prefill 的部署。

Ascend 910C

场景	P 配置 (GPU 数)	P tps/实例	D 配置 (GPU 数)	D tps/实例	P:D 比例	总 GPU 数
8K/4K	TP=8,EP=16,DP=2 (16)	26,861	TP=8,EP=16,DP=2 (16)	4,913	1P:1D	32
32K/4K	TP=8,EP=16,DP=2 (16)	24,818	TP=8,EP=32,DP=4 (32)	3,673	2P:1D	64
128K/4K	TP=8,EP=16,DP=2 (16)	18,863	TP=4,EP=32,DP=8 (32)	1,190	3P:1D	80
256K/4K	TP=8,EP=16,DP=2 (16)	14,291	TP=4,EP=32,DP=8 (32)	618	3P:1D	80

NVIDIA H20

场景	P 配置 (GPU 数)	P tps/实例	D 配置 (GPU 数)	D tps/实例	P:D 比例	总 GPU 数
8K/4K	TP=8,EP=8,DP=1 (8)	14,980	TP=8,EP=16,DP=2 (16)	4,303	1P:1D	24
32K/4K	TP=8,EP=8,DP=1 (8)	13,806	TP=4,EP=16,DP=4 (16)	3,272	2P:1D	32
128K/4K	TP=8,EP=8,DP=1 (8)	10,512	TP=4,EP=16,DP=4 (16)	1,457	5P:1D	56
256K/4K	TP=8,EP=8,DP=1 (8)	7,974	TP=4,EP=32,DP=8 (32)	1,711	14P:1D	144

4.4 910C vs H20 对比分析

跨平台对比揭示了清晰的模式：性能差距随上下文长度扩大，方向取决于阶段。

Prefill 吞吐量 (tps/gpu)

场景	910C	H20	H20/910C	910C GPU 数	H20 GPU 数
8K/4K	1,679	1,872	1.12x	16	8
32K/4K	1,551	1,726	1.11x	16	8
128K/4K	1,179	1,314	1.11x	16	8
256K/4K	893	997	1.12x	16	8

Decode 吞吐量 (tps/gpu)

场景	910C	H20	H20/910C	910C GPU 数	H20 GPU 数
8K/4K	307	269	910C 1.14x	16	16
32K/4K	115	205	H20 1.78x	32	16
128K/4K	37.2	91.1	H20 2.45x	32	16
256K/4K	19.3	53.5	H20 2.77x	32	32

Prefill 与 Decode 延迟

场景	910C Prefill (ms)	H20 Prefill (ms)	910C/H20	910C Decode (ms)	H20 Decode (ms)	H20/910C
8K/4K	325	547	1.68x	19.4	9.1	2.14x
32K/4K	1,327	2,345	1.77x	19.4	9.1	2.14x
128K/4K	6,914	12,328	1.78x	20.8	9.8	2.13x
256K/4K	18,255	32,581	1.78x	21.0	9.9	2.13x

Prefill 吞吐量：H20 在所有上下文长度下保持显著稳定的 1.11--1.12 倍领先，尽管仅使用 8 个 GPU 而 910C 使用 16 个。这意味着 H20 每集群实现了约 2.2 倍的吞吐——得益于其能以 EP=8（最小化

通信) 运行吞吐最优配置, 而 910C 需要 EP=16。H20 更高的 HBM 带宽也有助于在 Prefill 期间更有效地分摊内存受限的 mHC 开销。

Decode 吞吐量: 910C 在 8K/4K 下保持 1.14 倍的微弱领先——这是唯一赢得的上下文长度。这直接归因于 910C 的 EP 带宽优势: 在短上下文大 batch 下, MoE 通信主导 H20 Decode (57.3%, 见第 3.4 节), 而 910C 的 7.84 倍更快的 EP 链路将此劣势转化为净计算优势。然而在 32K 及以上, H20 决定性地领先 (1.78--2.77 倍), 因为不断增长的 KV 缓存读取和 Lightning Index 加载将瓶颈转向纯 HBM 带宽, H20 的 2.22 倍优势不断累积。

延迟: 910C 实现 1.68--1.78 倍更快的 Prefill 延迟 (计算受限的矩阵乘法优势), 而 H20 实现 2.13--2.14 倍更快的 Decode (内存带宽受限优势)。两个比率在不同上下文长度下保持稳定, 证实决定延迟差距的是根本性的硬件特征——而非工作负载形态。

5. 关键模块分析

5.1 mHC 优化

manifold Hyper Connection (mHC) 是 V4 最具特色——也是最昂贵——的架构组件。hc_mult=4 意味着每个层边界处激活从 hidden_size (4,096) 扩展到 4 倍 (16,384), 为注意力前和 MoE 后的混合操作产生巨大的 HBM 流量。未经优化时, mHC 消耗 84.6% 的 Prefill 时间。我们分析了四个逐步激进的优化级别:

优化级别

级别	mhc_kernel_fused	mhc_sp	mhc_fused_bf16	描述
未融合 FP32	False	False	False	原始基线
融合 FP32	True	False	False	内核融合 (新默认值)
融合 FP32 + SP	True	True	False	+ mHC 序列并行
融合 BF16 + SP	True	True	True	+ 融合算子采用 BF16 精度

Prefill 时间对比 (910C, TP=8, EP=16, DP=2, BS=16)

级别	8K/4K	mHC %	32K/4K	mHC %	相对未融合加速比 (8K)
未融合 FP32	10,110 ms	84.6%	41,188 ms	83.0%	1.00x

级别	8K/4K	mHC %	32K/4K	mHC %	相对未融合加速比（8K）
融合 FP32	2,458 ms	36.5%	10,579 ms	33.9%	4.11x
融合 FP32 + SP	1,673 ms	6.7%	7,437 ms	6.0%	6.04x
融合 BF16 + SP	1,617 ms	3.5%	7,213 ms	3.1%	6.25x

仅内核融合的效果就极为显著：8K 下 4.11 倍加速，将 mHC 从总 Prefill 时间的 84.6% 降至 36.5%。融合消除了独立的 sinkhorn 迭代内核（20 次类 softmax 归一化迭代）和前/后混合内核，将它们合并为单次内核执行，HBM 流量减少约 10 倍。为 mHC 添加序列并行（mHC-SP）将 4 倍扩展的激活分布在 TP rank 间，进一步带来 1.47 倍加速（总计 6.04 倍）。最终的 BF16 优化将融合操作的每元素字节数减半，但增量收益有限（总计 6.25 倍，仅比 Fused+SP 提升 3.5%），因为剩余的 mHC 开销已经很小，仅占 3.5%。

瓶颈迁移（910C，8K/4K）

类别	未融合 FP32	融合 FP32	融合+SP	融合 BF16+SP
mHC	84.6%	36.5%	6.7%	3.5%
注意力投影	5.6%	23.0%	33.8%	35.0%
通信	4.6%	18.8%	27.6%	28.6%
注意力计算	1.1%	4.5%	6.6%	6.8%
Lightning Index	1.1%	4.5%	6.7%	6.9%
KV 压缩	1.8%	7.4%	10.8%	11.2%
MoE（Gate+Routed）	0.7%	3.0%	4.5%	4.6%
Norm	0.1%	0.3%	0.5%	0.5%
Embedding/LMHead	0.5%	1.9%	2.8%	2.8%

瓶颈迁移表展示了逐步优化暴露下一瓶颈的教科书式案例。随着 mHC 从 84.6% 降至 3.5%，注意力投影从 5.6% 上升至 35.0%，通信从 4.6% 上升至 28.6%，KV 压缩从 1.8% 上升至 11.2%。在完全优化的 Fused BF16+SP 配置下，工作负载在多个类别间趋于均衡，表明进一步提升需要优化注意力投影或通信，而非仅优化 mHC。

SP/mHC-SP 对比（融合基线，TP=8，EP=16，DP=2）

场景	无 SP	仅 SP	SP + mHC-SP	加速比（无 SP -> SP+mHC-SP）
8K/4K (910C)	3,474 ms	2,458 ms	1,673 ms	2.08x
32K/4K (910C)	14,668 ms	10,579 ms	7,437 ms	1.97x
128K/4K (910C)	71,981 ms	55,599 ms	43,030 ms	1.67x
256K/4K (910C)	179,522 ms	146,748 ms	121,611 ms	1.48x
8K/4K (H20)	11,681 ms	4,341 ms	3,988 ms	2.93x
32K/4K (H20)	48,196 ms	18,824 ms	17,410 ms	2.77x
128K/4K (H20)	216,574 ms	99,072 ms	93,416 ms	2.32x
256K/4K (H20)	496,622 ms	261,612 ms	250,300 ms	1.98x

SP 和 mHC-SP 共同带来显著的 Prefill 加速，但在更长序列下收益递减。在 910C 的 8K 下，SP+mHC-SP 组合相比无 SP 实现 2.08 倍加速；在 256K 下降至 1.48 倍，因为 SP 的收益——减少每 rank 激活内存——被更长序列下增加的 AllGather 通信所抵消。H20 从 SP 中获益更多（8K 下高达 2.93 倍），因为其更高的 HBM 带宽使释放的内存带宽更有价值，且更快的 TP 链路（450 GB/s）保持了较低的 AllGather 开销。仅 SP 与 SP+mHC-SP 之间的差距在 H20 上为 8.1%（4,341 vs 3,988 ms），而在 910C 上为 32%（2,458 vs 1,673 ms），证实 910C 从 mHC-SP 中不成比例地获益，因为其较低的 HBM 带宽使 mHC 内存流量成为更大的相对瓶颈。

5.2 注意力与 KV 缓存分析

V4 的 KV 压缩是对部署经济性影响最大的架构创新，能在固定 HBM 预算内实现显著更大的 batch 规模和更长的上下文窗口。

KV 缓存随序列长度变化（910C，TP=4，EP=32，DP=8，BS=8）

序列长度	V4 KV 缓存	无压缩	V3 MLA	相对无压缩节省倍数	Decode 步时间
1K	0.013 GB	0.090 GB	0.072 GB	7.1x	18.0 ms
2K	0.020 GB	0.180 GB	0.144 GB	9.1x	18.0 ms
4K	0.034 GB	0.361 GB	0.288 GB	10.7x	19.4 ms
8K	0.062 GB	0.721 GB	0.576 GB	11.6x	19.4 ms
16K	0.118 GB	1.443 GB	1.151 GB	12.2x	19.4 ms

序列长度	V4 KV 缓存	无压缩	V3 MLA	相对无压缩节省倍数	Decode 步时间
32K	0.231 GB	2.886 GB	2.303 GB	12.5x	19.4 ms
64K	0.457 GB	5.771 GB	4.605 GB	12.6x	19.5 ms
128K	0.907 GB	11.543 GB	9.211 GB	12.7x	20.8 ms

KV 缓存随序列长度变化 (H20, TP=4, EP=32, DP=8, BS=8)

序列长度	V4 KV 缓存	无压缩	V3 MLA	相对无压缩节省倍数	Decode 步时间
1K	0.013 GB	0.090 GB	0.072 GB	7.1x	8.4 ms
2K	0.020 GB	0.180 GB	0.144 GB	9.1x	8.4 ms
4K	0.034 GB	0.361 GB	0.288 GB	10.7x	9.1 ms
8K	0.062 GB	0.721 GB	0.576 GB	11.6x	9.1 ms
16K	0.118 GB	1.443 GB	1.151 GB	12.2x	9.1 ms
32K	0.231 GB	2.886 GB	2.303 GB	12.5x	9.1 ms
64K	0.457 GB	5.771 GB	4.605 GB	12.6x	9.1 ms
128K	0.907 GB	11.543 GB	9.211 GB	12.7x	9.8 ms

V4 的 KV 缓存由于异构压缩调度而呈亚线性增长。在 8K 下，V4 每 batch 元素仅需 0.062 GB——而未压缩需 0.721 GB，V3 MLA 需 0.576 GB——压缩比为 11.6 倍。在 128K 下该比率提升至 12.7 倍，因为 C128A 层（128 倍压缩）在更长序列中贡献比例更大。两个平台上 Decode 步时间保持几乎恒定（910C 19.4 ms，H20 9.1 ms），从 1K 到 32K 均是如此，仅在 128K（20.8 ms / 9.8 ms）时因 Lightning Index 缓存读取变得不可忽略才有所增加。在 H20 上，即使 64K 下 Decode 仍保持 9.1 ms——4,000 GB/s 带宽吸收了额外的 KV 读取而无明显延迟影响。

各层类型 KV 缓存分布

序列长度	C4A (%)	C128A (%)	总量 (GB)
8K	93.2%	6.3%	0.062
32K	96.5%	3.4%	0.231
128K	97.4%	2.6%	0.907

C4A 层主导 KV 缓存（93.2--97.4%），尽管 C128A 层数量几乎相当（20 vs 21）。这是因为 C128A 的 128 倍压缩将每层的 KV 贡献缩减到极小比例。在 128K 下，20 个 C128A 层总共仅贡献 23.6

MB，而 21 个 C4A 层贡献 883.6 MB。2 个全注意力层（SWA，window_size=128）在此模型中不存储持久 KV 缓存，因为其窗口太小，相对于压缩层所需的存储可以忽略不计。

压缩与未压缩对比

序列长度	V4 压缩	V4 未压缩	V3 MLA	V4 vs 未压缩	V4 vs V3
8K	0.062 GB	0.721 GB	0.576 GB	11.6x	小 9.3 倍
32K	0.231 GB	2.886 GB	2.303 GB	12.5x	小 10.0 倍
64K	0.457 GB	5.771 GB	4.605 GB	12.6x	小 10.1 倍
128K	0.907 GB	11.543 GB	9.211 GB	12.7x	小 10.2 倍

V4 压缩后的 KV 缓存比 V3 MLA 小 9.3--10.2 倍。这是一个变革性的改进：在 128K 上下文、BS=8 下，V4 的 KV 缓存仅需 0.907 GB，而 V3 需要 9.211 GB——为每 rank 释放 8.3 GB 的 HBM 用于额外 batch 容量或模型权重。实际而言，这使 V4 能在相同内存预算下多服务 10 倍的并发请求，或扩展到 10 倍更长的上下文。压缩比随序列长度略有提升（从 8K 的 9.3 倍到 128K 的 10.2 倍），因为高压缩的 C128A 层在有效缓存中占比越来越大。

注意力计算随序列长度变化（910C，每层，TP=8，EP=16，BS=16）

层类型	8K Attn (ms)	8K Attn%	32K Attn (ms)	32K Attn%	128K Attn (ms)	128K Attn%
Full (ratio=1)	11.7	25.0%	46.8	25.1%	187.2	25.1%
C4A (ratio=4)	14.6	24.0%	58.5	21.1%	233.9	13.9%
C128A (ratio=128)	12.4	24.3%	52.6	25.5%	280.7	31.4%

注意力计算随序列长度变化（H20，每层，TP=8，EP=8，BS=16）

层类型	8K Attn (ms)	8K Attn%	32K Attn (ms)	32K Attn%	128K Attn (ms)	128K Attn%
Full (ratio=1)	29.7	40.0%	118.9	40.1%	475.5	40.1%
C4A (ratio=4)	37.1	34.6%	148.6	30.4%	594.3	20.4%
C128A (ratio=128)	30.6	35.3%	133.7	37.3%	713.2	44.3%

注意力计算扩展分析揭示了一个反直觉的模式：尽管 C128A 层有 128 倍 KV 压缩，它们在长上下文下却成为最昂贵的层。在 910C 的 128K 下，C128A 注意力每层耗时 280.7 ms（占层时间的 31.4%），而 C4A 为 233.9 ms（13.9%），全注意力为 187.2 ms（25.1%）。这是因为 C128A 层使用 Lightning Index 从完整压缩缓存中选择 top-512 条目，索引分数计算（64 头 * 128 dim * 完整序列长度）随序列长度线性增长，与压缩比无关。C4A 层直接读取 4 倍压缩的缓存，注意力开销更低，因为其有效缓存更小且无需基于索引的检索。

在 H20 上这一效应更为明显：C128A 注意力在 128K 下达到 713.2 ms（占层时间的 44.3%）。H20 每次注意力操作耗时是 910C 的 2.54 倍（713.2 vs 280.7 ms），因为注意力涉及的矩阵乘法在 Prefill 期间是 CUBE 受限的，而 H20 的矩阵乘法吞吐低 2.54 倍。全注意力层在 910C 上保持稳定的约 25.1% 占比，在 H20 上保持 40.1%——其 SWA 窗口（128 token）限定了开销上限，与总上下文无关。

6. 部署建议

6.1 短上下文（8K/4K）——对话/编程

短上下文工作负载（对话、编程助手）优先考虑低延迟和高单 GPU 吞吐。在 8K/4K 下，两个平台的 P:D 比例均为 1:1，意味着 Prefill 和 Decode 需要等量的 GPU 资源。这是 GPU 效率最高的运行点。

平台	Prefill	Decode	P:D	总 GPU 数
910C	TP=8, EP=16, DP=2 (16 GPUs)	TP=8, EP=16, DP=2 (16 GPUs)	1:1	32
H20	TP=8, EP=8, DP=1 (8 GPUs)	TP=8, EP=16, DP=2 (16 GPUs)	1:1	24

910C（32 GPU）：Prefill 和 Decode 使用相同的 TP=8、EP=16、DP=2 配置。这种对称配置简化了部署——单一集群配置即可处理两个阶段。910C 的 Decode 吞吐为 307 tps/gpu，比 H20 高 1.14 倍，得益于其 EP 带宽优势使 MoE 调度保持快速。325 ms 的 Prefill 延迟满足交互式对话需求。

H20（24 GPU）：通过仅使用 8 个 GPU 进行 Prefill（EP=8 最小化通信）同时将 Decode 扩展到 16 个 GPU，实现了更高 GPU 效率的部署。尽管总 GPU 数比 910C 少 25%，H20 的 Prefill 吞吐达到 1,872 tps/gpu（910C 的 1.12 倍）。代价是更高的 Prefill 延迟（547 ms vs 325 ms），对于面向吞吐的对话服务来说可以接受。

6.2 中等上下文（32K/4K）——文档处理

在 32K 上下文下，P:D 比例升至 2:1，Prefill GPU 需求相对 Decode 翻倍。这反映了 8 倍更大的输入（32K vs 4K 输出），使 Prefill 成为吞吐瓶颈。

平台	Prefill	Decode	P:D	总 GPU 数
910C	TP=8, EP=16, DP=2 (16 GPUs)	TP=8, EP=32, DP=4 (32 GPUs)	2:1	64
H20	TP=8, EP=8, DP=1 (8 GPUs)	TP=4, EP=16, DP=4 (16 GPUs)	2:1	32

910C（64 GPU）：Decode 转为 TP=8、EP=32、DP=4（32 GPU），以处理更大的 KV 缓存（32K 下 0.231 GB vs 8K 下 0.062 GB）。更宽的 EP=32 将专家权重分布到更多 rank，降低每 rank 内存压力。Prefill 配置保持不变（TP=8、EP=16、DP=2）。Decode 吞吐降至 115 tps/gpu——不断增长的 KV 缓存和 Lightning Index 读取开始占据主导。

H20（32 GPU）：Prefill 维持高效的 EP=8 配置。Decode 使用 TP=4、EP=16、DP=4，达到 205 tps/gpu——910C Decode 吞吐的 1.78 倍——因为 H20 的 4,000 GB/s HBM 带宽能高效服务更大的缓存读取。总 GPU 数恰好是 910C 的一半（32 vs 64），使 H20 在此上下文长度下性价比显著更优。

6.3 长上下文（128K/4K）——RAG/文档问答

长上下文 RAG 和文档问答工作负载进一步拉大 P:D 比例。Lightning Index 成为 Prefill 的主要开销（910C 上 30%），KV 缓存在 128K 下达到每 batch 元素 0.907 GB。

平台	Prefill	Decode	P:D	总 GPU 数
910C	TP=8, EP=16, DP=2 (16 GPUs)	TP=4, EP=32, DP=8 (32 GPUs)	3:1	80
H20	TP=8, EP=8, DP=1 (8 GPUs)	TP=4, EP=16, DP=4 (16 GPUs)	5:1	56

910C（80 GPU，P:D=3:1）：Decode 转为 TP=4、EP=32、DP=8（32 GPU）——比 Prefill 更低的 TP，因为 Decode 的内存受限特性从额外 TP 并行中收益有限。910C 的 Decode 吞吐降至 37.2 tps/gpu，但其 EP 带宽优势使整体 P:D 比例保持在适度的 3:1。

H20（56 GPU，P:D=5:1）：H20 的 P:D 比例在 128K 下跳升至 5:1，需要 5 个 Prefill 实例（40 GPU）对应 1 个 Decode 实例（16 GPU）。尽管 H20 的 Decode 吞吐更优（91.1 tps/gpu，910C 的 2.45 倍），其更小的 Prefill 实例（8 GPU）无法足够快地产出 token 以跟上节奏。这凸显了一个关键部署约束：在 H20 上，长上下文服务越来越受 Prefill 瓶颈限制。

6.4 超长上下文（256K/4K）——全文档分析

超长上下文是最具挑战性的部署场景，Prefill 时间在 910C 上超过 18 秒，在 H20 上超过 32 秒。

平台	Prefill	Decode	P:D	总 GPU 数
910C	TP=8, EP=16, DP=2 (16 GPUs)	TP=4, EP=32, DP=8 (32 GPUs)	3:1	80
H20	TP=8, EP=8, DP=1 (8 GPUs)	TP=4, EP=32, DP=8 (32 GPUs)	14:1	144

910C（80 GPU，P:D=3:1）：910C 维持与 128K 相同的配置，P:D 比例仍可控在 3:1。18.3 秒的 Prefill 延迟较高但对于批量文档处理来说可接受。910C 的总 GPU 数保持在 80 个，与 128K 相同，因为 P:D 比例向上取整为相同的值（2.77 取整为 3）。

H20（144 GPU，P:D=14:1）：H20 的 P:D 比例飙升至 14:1，需要 112 个 GPU 用于 Prefill，32 个用于 Decode。这种极端失衡源于长输入（256K token）、H20 计算受限的 Prefill（32.6 秒延迟）和相对高效的 Decode（32 GPU 上 53.5 tps/gpu）的组合。在此规模下，H20 的 GPU 成本是 910C 的 1.8 倍（144 vs 80 GPU），使 910C 在超长上下文下更具成本效益，尽管 H20 的单 GPU 性能更优。运营商应仔细评估 256K 上下文是否真正必要，还是可以通过 128K 分块处理满足质量需求。

6.5 通用指导

按工作负载选择平台：

- 短上下文（8K 以内），Decode 吞吐关键型：910C 具有竞争力或更优（8K 下 Decode 吞吐优势 1.14 倍），得益于其 EP 带宽。当 MoE 通信成为瓶颈时选择 910C。
- 中长上下文（32K--128K），吞吐关键型：H20 提供更优的 Decode 吞吐（1.78--2.45 倍）且所需 GPU 更少。HBM 带宽优势在 KV 缓存读取成为瓶颈时占主导。
- 延迟关键型 Prefill：910C 持续实现 1.68--1.78 倍更快的 Prefill。对于首 token 延迟至关重要的交互式应用，优选 910C。
- 超长上下文（256K）：强烈推荐 910C——80 GPU vs H20 的 144 GPU，可控的 3:1 P:D 比例 vs 14:1。

关键优化杠杆：

- mHC 内核融合是必须的。未融合时 mHC 消耗 84.6% 的 Prefill 时间。融合 BF16 内核 + SP 将其降至 3.5%，带来 6.25 倍端到端加速。
- 应启用 SP 和 mHC-SP。Prefill 综合加速 1.5--2.1 倍。在短上下文下收益更大。
- KV 压缩是 V4 的决定性优势。相比 V3 MLA 缩减 9.3--10.2 倍，使在相同 HBM 预算下支持 10 倍更大的 batch 或 10 倍更长的上下文。

- 长上下文下 P/D 分离至关重要。在 128K+ 下，使用不同 TP/EP 配置的独立 Prefill 和 Decode 集群，避免统一配置在两个阶段都妥协带来的资源浪费。

7. 行业启示

7.1 KV 缓存管理与分层缓存

V4 的异构 KV 压缩——相对未压缩缓存实现 9.3--12.7 倍缩减，相比 V3 MLA 缩减 4.6 倍——从根本上改变了 KV 缓存管理格局。在 128K 上下文、BS=8 下，V4 每 rank 仅需 0.907 GB KV 缓存，而未压缩需 11.543 GB，释放超过 10 GB 的 HBM 用于额外 batch 容量。

这种压缩与正在进入生产的分层 KV 缓存系统形成强大协同。LMCache 和 NVIDIA Dynamo 等框架将 KV 存储从 GPU HBM 扩展到 CPU DRAM 和 SSD，通过利用跨请求的缓存复用实现高达 15 倍的吞吐提升和 2 倍的延迟降低。V4 的压缩 KV 表示使分层缓存更具吸引力：8K 下每 batch 元素仅 0.062 GB 完全可以放入适度的 GPU 端缓存层，而更长上下文条目（128K 下 0.907 GB）可以在 DRAM 和 SSD 层间高效调度，带宽开销极小。异构压缩调度意味着缓存管理策略应感知压缩比：C4A 层贡献 93.2--97.4% 的 KV 缓存，是淘汰和预取决策的主要目标，而 C128A 条目如此之小（总量的 2.6--6.3%）以至于可以无限期地保留在最快层。

行业趋势正朝向以 KV 缓存为中心的服务架构——Mooncake（FAST'25 最佳论文）和 llm-d 项目的缓存感知路由即是代表——这与 V4 的设计天然契合。在架构层面压缩 KV 缓存（而非依赖训后量化）的模型，能提供可预测的缓存大小和访问模式，供分层存储系统优化。

7.2 P/D 分离架构

我们的 P:D 比例分析揭示了一个关键的部署规划洞察：Prefill 与 Decode 集群之间的 GPU 分配随上下文长度和硬件平台剧烈变化。在 910C 上，比例从 8K 的 1:1 适度增长到 256K 的 3:1。在 H20 上，则在 256K 时飙升至 14:1，每 32 个 Decode GPU 需要 112 个 Prefill GPU。

这些发现验证了行业对 P/D 分离的快速采纳，该技术从研究（DistServe, OSDI'24）到生产部署仅用不到 18 个月。主要推理框架——NVIDIA Dynamo、vLLM 和 SGLang——现已原生支持分离式服务，Meta、LinkedIn 和 Mistral 已在生产中运行。核心收益是避免统一配置在计算密集型 Prefill 和内存密集型 Decode 之间妥协带来的资源浪费。

V4 的架构使分离尤为有价值，因为其 Prefill 和 Decode 特性差异巨大。Prefill 由 CUBE 受限的矩阵乘法操作主导（注意力投影 23.2%，Lightning Index 在 256K 下增至 44.4%），而 Decode 由

MEM 受限的权重读取（MoE 路由专家 34.7--46.6%）和 COMM 受限的调度（14.3--57.3%，取决于平台）主导。对两个阶段使用相同的硬件配置，要么在 Decode 中浪费 910C 的计算优势，要么在 Prefill 中浪费 H20 的带宽优势。

H20 在 256K 下的极端 14:1 比例还凸显了一个新兴挑战：Prefill 与 Decode 工作节点之间的 KV 缓存传输。在 256K 上下文下，压缩后的 KV 缓存约为每请求 7.3 GB（0.907 GB * 8 batch 元素），必须在请求路由时传输。使用 H20 的 50 GB/s EP 带宽，此传输约需 146 ms——为首 token 延迟增加了不可忽略的开销。这推动了“以 KV 缓存为中心”的分离架构，其中 KV 数据的局部性驱动请求路由。

7.3 硬件设计权衡

910C 与 H20 的对比揭示了推理加速器硬件设计理念的深刻启示。910C 的 2.54 倍计算优势（376 vs 148 TFLOPS BF16）带来了一致的 1.68--1.78 倍 Prefill 延迟优势，而 H20 的 2.22 倍内存带宽优势（4,000 vs 1,800 GB/s）带来了 2.13--2.14 倍的 Decode 延迟优势。任何一个平台都无法在所有工作负载上占据主导。

更深层的洞察在于 Cube:Vec 比率。910C 的 15.7:1（376 TFLOPS / 24 TFLOPS）意味着向量化操作（如 mHC 的逐元素混合）相对于矩阵乘法受到严重惩罚。这就是为什么 mHC 消耗 910C Prefill 时间的 36.8%，而在 H20 上（Cube:Vec = 3.4:1）仅占 9.2%——H20 更均衡的架构能更高效地处理矩阵乘法/向量混合作负载。随着模型越来越多地采用非标准连接模式（超级连接、状态空间混合器、MoE 门控），具有均衡计算特性的硬件可能比峰值 FLOPS 优化的设计更具适应性。

EP 带宽差异（392 vs 50 GB/s，7.84 倍）是影响最大的单一硬件差异。它将 MoE 从通信瓶颈（H20 Decode 的 57.3%）转变为可控开销（910C 上 21.6%）。这表明未来的推理加速器应优先实现节点间带宽与节点内带宽的对等，特别是对于拥有数百专家的 MoE 架构。910C 通过对 TP 和 EP 流量使用相同的高带宽互联来实现这一点，这一设计选择在 256 专家 MoE 上获得了丰厚回报。

7.4 MoE 的网络带宽需求

V4 的 256 专家 MoE 调度在每一层都产生大量 all-to-all 通信：在 EP=16 下，每个 token 必须路由到 16 个 rank 中的 6 个，产生的调度和合并流量与 $\text{batch_size} * \text{激活专家数} * \text{hidden_size}$ 成正比。在 H20 上，此通信在 8K/4K 下主导 Decode，占步时间的 57.3%，而在 910C 上仅占 21.6%。

这一差距对数据中心网络设计有直接影响。在 EP=16 下，H20 需要 800 GB/s 的二分带宽（16 rank * 50 GB/s 每链路）以避免 EP 通信成为瓶颈。扩展到 EP=32 会使此需求翻倍。近期关于数据中心级高带宽域的研究（InfiniteHBD, SIGCOMM 2025）表明，光电路交换可以为 EP 流量提供 Tbps 级别

的带宽，但此类基础设施的成本必须与使用更少但互联更快的 GPU（如 910C）的替代方案相权衡。

MegaScale-MoE 将 SP+EP 并行相结合的方法（实现比 TP+TP 高 14.9--32.9% 的 MFU）与我们的发现一致：910C 的吞吐最优配置使用 EP=16 配合 SP 启用，而 H20 将 EP 最小化到 8 以避免通信开销。对基础设施规划者的实际建议：拥有 256+ 专家的 MoE 模型需要（a）高带宽节点间链路（每 GPU > 200 GB/s，如 910C），或（b）保守的 EP 配置（EP=8），以牺牲并行度来适应网络带宽限制。

7.5 mHC 作为新范式

V4 的 manifold Hyper Connection (mHC) 用 4 倍扩展的混合操作替代标准残差连接。未经内核融合时，仅这一架构选择就消耗了 84.6% 的 Prefill 时间——超过所有其他操作的总和。仅融合带来的 4.11 倍加速，以及融合 + SP + BF16 的 6.25 倍加速，表明 mHC 只有在配合激进的系统级优化时才具有实用性。

这创造了模型-系统协同设计的新模式：提供训练收益（更好的梯度流动、改善的跨层信息共享）的架构创新可能带来严重的推理开销，除非同步开发内核级优化。mHC 的经验表明，模型架构师在提出新连接模式时不仅应评估训练损失，还应评估推理 Roofline 特征。

瓶颈迁移分析（第 5.1 节）展示了一个普遍原则：将一个组件优化到接近零只会暴露下一个瓶颈。随着 mHC 从 84.6% 降至 3.5%，注意力投影从 5.6% 上升到 35.0%，通信从 4.6% 上升到 28.6%。这种“打地鼠”模式是成熟推理优化的典型特征——没有银弹，只有逐步的瓶颈消除。

近期的 mHC 论文（Manifold-Constrained Hyper-Connections）通过将残差连接空间约束到特定流形上来解决原始方法的稳定性和可扩展性问题。这表明架构社区已经在迭代这一概念，未来的变体可能在保持梯度流动收益的同时减少 4 倍扩展因子。

7.6 超长上下文服务

我们对 128K 和 256K 上下文场景的分析揭示了行业在长上下文模型成为标准时必须解决的若干实际挑战：

Prefill 延迟变为面向用户的。在 910C 的 256K 下，最优 Prefill 延迟为 18.3 秒（64 GPU）。在 H20 上达到 32.6 秒。这些延迟对于批量文档处理尚可容忍，但对交互式应用不可接受。行业的应对——分块 Prefill、推测解码和前缀缓存——只能部分缓解，因为 V4 的异构压缩调度意味着不同的块有不同的计算特性。

Lightning Index 在长序列下占据主导。索引分数计算随序列长度线性增长（每层 64 头 * 128 dim * 完整序列长度），导致 Lightning Index 在 910C 上从 8K 的 Prefill 时间 4.4% 上升到 256K 的 44.4%（H20 上为 42.5%）。这是长上下文 V4 服务最显著的架构瓶颈。不同于通过内核融合驯化的 mHC，Lightning Index 涉及的全精度矩阵乘法操作已经在硬件 Roofline 上运行——没有容易获取的 4 倍加速等待解锁。可能需要近似 top-K 选择或分层索引。

带宽受限硬件上的 P/D 比例爆炸。H20 的 P:D 比例在 256K 下达到 14:1（共 144 GPU），而 910C 的可控 3:1（80 GPU）。这意味着超长上下文在带宽受限平台上需要严重偏向 Prefill 的集群，78% 的 GPU 专用于 Prefill。这种低效可能促使运营商在长上下文工作负载上转向计算充裕的平台，即使带宽充裕的平台提供更好的单 GPU Decode 吞吐。

Batch 规模限制。在 256K 下，即使有 V4 的激进 KV 压缩，Prefill 吞吐配置的最大 batch 规模也降至 16（910C）和 8（H20）。910C 的 64 GB HBM 容纳 16 请求 * 0.907 GB KV 缓存 + 24 GB 权重，几乎没有余量。如果没有 V4 的压缩（128K 下每请求需 11.5 GB KV 缓存），即使单个 256K 请求也将消耗 23 GB 的 KV 缓存——几乎是 910C HBM 的一半。V4 相对 V3 的 4.6 倍 KV 压缩使 256K 服务成为可能，但 batch 规模仍然太小，无法在面向吞吐的部署中实现高效的 GPU 利用率。

8. 附录

8.1 硬件与模型参数

所有硬件和模型参数均来源于公开规格和配置文件。以下表格记录了本报告所有性能模型计算中使用的确切数值。利用率（50% 计算，80% HBM 带宽）代表生产推理工作负载的保守估计。

Ascend 910C:

参数	值
Cube TFLOPS (BF16)	376
Vec TFLOPS (FP32)	24
HBM 容量	64 GB
HBM 带宽	1,800 GB/s
TP 带宽	392 GB/s
EP 带宽	392 GB/s
网络延迟	10 us

参数	值
Flops 利用率	50%
HBM BW 利用率	80%
BW 利用率	80%

NVIDIA H20:

参数	值
Cube TFLOPS (BF16)	148
Vec TFLOPS (FP32)	44
HBM 容量	96 GB
HBM 带宽	4,000 GB/s
TP 带宽	450 GB/s
EP 带宽	50 GB/s
网络延迟	5 us
Flops 利用率	50%
HBM BW 利用率	80%
BW 利用率	80%

DeepSeek V4 模型配置:

参数	值
架构	DeepseekV4ForCausalLM
Hidden size	4,096
Layers	43
Q heads	64
KV heads	1 (MQA)
Head dim	512
RoPE head dim	64
Q LoRA rank	1,024

参数	值
O groups	8
O LoRA rank	1,024
Routed experts	256
Activated experts	6
Shared experts	1
MoE inter dim	2,048
HC mult	4
HC Sinkhorn iters	20
Window size	128
Index heads	64
Index head dim	128
Index topK	512
Vocab size	129,280
Compress ratios	2 full + 21 C4A + 20 C128A
Dtype	bfloat16

运行时默认参数：

参数	值
seq_len	8,192
batch_size	8
output_len	4,096
TP	8
EP	64
DP	8
SP	True
mhc_kernel_fused	True
shared_expert_overlapped	True

参数	值
moe_load_balance_factor	1.2

8.2 方法论

性能模型。所有结果由基于 Roofline 的分析性能模型（`perf_model/`）产出。对于每个操作，模型独立计算三个开销估计：CUBE 时间（矩阵乘法 FLOPs / 有效 TFLOPS）、VEC 时间（逐元素 FLOPs / vec TFLOPS）和 MEM 时间（传输字节数 / 有效 HBM 带宽）。瓶颈由 `argmax(cube, vec, mem)` 确定，操作总时间为 `max(cube, vec, mem) + communication_time`。所有大小假设 BF16（每元素 2 字节），FLOPs 遵循约定 $[M,K] \times [K,N]$ 矩阵乘法 = $M * N * K * 2$ 。

通信模型。TP 通信（AllReduce、AllGather、ReduceScatter）使用基于环的算法和 TP 带宽建模。EP 通信（MoE 调度/合并的 AllToAll）使用 EP 带宽。通信和计算建模为顺序执行（无重叠），提供保守的上界估计。

参数搜索。最优配置通过穷举网格搜索找到：TP 属于 {1,2,4,8,16,32,64}，EP 属于 {8,16,32,64,128,256}，DP 属于 {1,2,4,8}，batch_size 属于 {1..512}，seq_len 属于 {1K..32K}。约束 $(TP * DP) \% EP == 0$ 确保有效划分。每个配置在计算性能指标前先评估 HBM 容量可行性。

Decode 快速模式。Decode 模型评估使用周期采样配合梯形插值：每步开销分解为常数 + 线性(S) + 周期(S)，其中周期 $P = \text{LCM}(\text{compress_ratios}) = 128$ 。仅采样首尾各 P 步，然后通过 $T_{\text{total}} = N * (T_{\text{first}} + T_{\text{last}}) / (2P)$ 插值。对于 output_len=4096 实现 16 倍加速，误差低于 0.001%。

数据生成。所有表格和图表由 `report/analyze_scenarios.py` 生成，该脚本在所有场景组合上运行性能模型，并将结构化 JSON 数据导出到 `report/data/`。无人工数据录入——所有数字均可追溯到分析模型。

8.3 数据来源文件

所有原始数据位于 `report/data/`：

- `search_results_910C.json` / `search_results_H20.json` —— 各场景 top-20 配置（4 种场景：8K、32K、128K、256K）
- `pd_ratio_analysis.json` —— P/D 比例计算
- `op_analysis.json` —— 算子级瓶颈分析
- `sp_comparison.json` —— SP/mHC-SP 对比
- `mhc_optimization_comparison.json` —— 4 个 mHC 优化级别
- `hardware_comparison.json` —— 跨平台对比

- `v3_comparison.json` —— V4 vs V3 架构对比
- `kv_cache_scaling.json` —— KV 缓存随序列长度变化
- `attention_analysis.json` —— 各层类型注意力分析